

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524358
Name	Shaik .Abdul Razak

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I Shaik .Abdul Razak(192524358) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Shaik .Abdul Razak

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

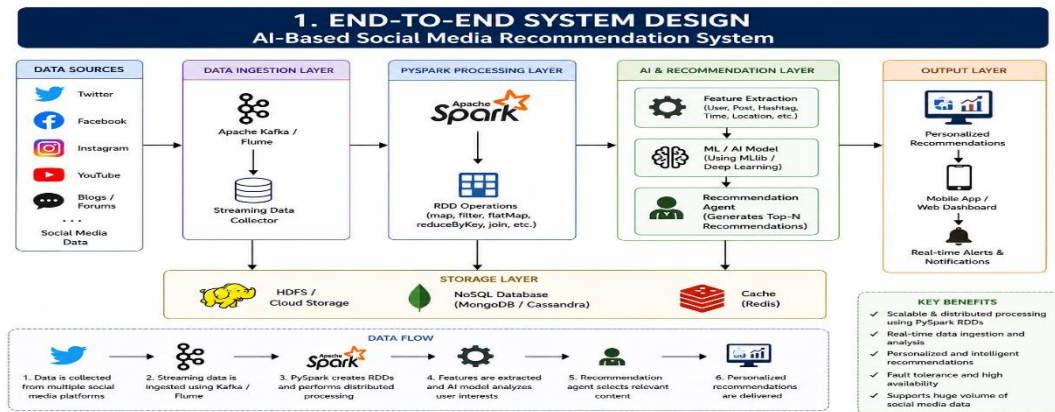
Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. End-to-End System Design

Proposed Solution: AI-Based Social Media Recommendation System

The proposed system collects large volumes of social media data and processes them using PySpark RDDs. The system identifies user interests, analyzes posts, and generates personalized recommendations. An intelligent recommendation agent uses the processed information to select suitable content for each user.



Data Flow

1. Social media posts, comments, hashtags, and user interactions are collected.
2. PySpark creates distributed RDDs from the incoming data.
3. filter() removes invalid or unwanted records.
4. map() converts raw records into useful features.
5. reduceByKey() calculates statistics such as hashtag or topic frequency.
6. The AI model identifies user interests.
7. The recommendation agent selects relevant content.
8. Results are displayed to users through an application or dashboard.

Intelligent Agent

The recommendation agent observes user activity, analyzes interests, and selects appropriate content. It can continuously update recommendations when new social media information becomes available.

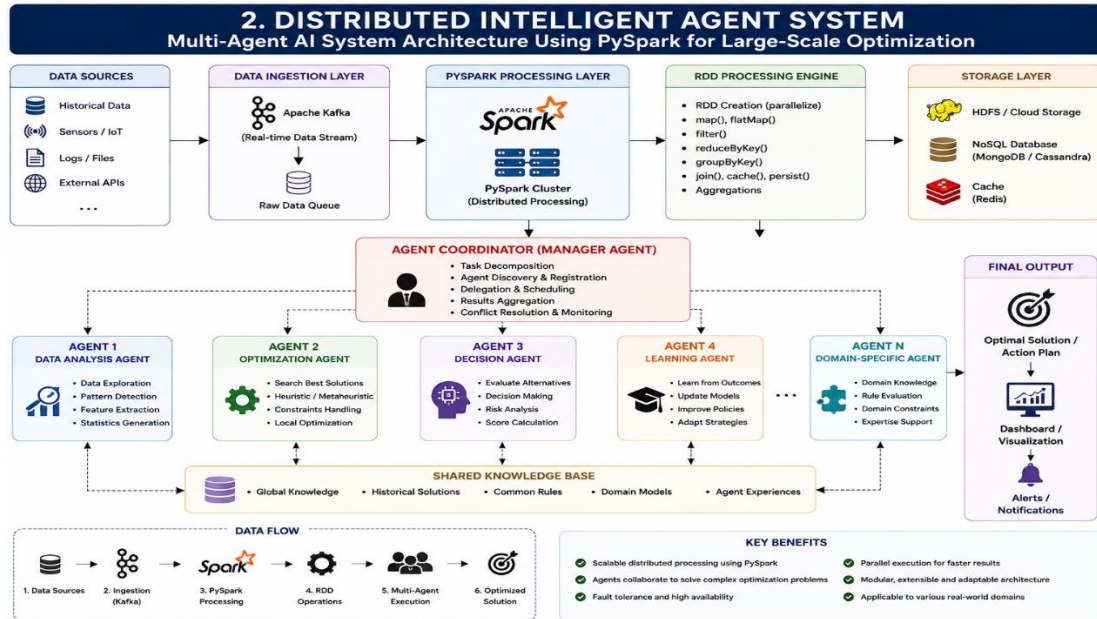
Expected Result

The system provides personalized recommendations while distributing large-scale processing across multiple Spark nodes. PySpark improves processing speed and RDD fault tolerance ensures that lost partitions can be recomputed.

2. Distributed Intelligent Agent System

Proposed Solution: Multi-Agent Optimization System

This system uses multiple intelligent agents to solve a large-scale optimization problem. PySpark distributes the data processing among worker nodes, while individual agents perform specialized tasks.



Working

1. The input dataset is divided into partitions.
2. PySpark distributes the partitions among worker nodes.
3. Different agents analyze different parts of the problem.
4. The analysis agent extracts useful information.
5. The optimization agent searches for better solutions.
6. The decision agent evaluates candidate solutions.
7. The coordinator combines the results from all agents.
8. The best solution is returned to the user.

Example

The system can be applied to **delivery route optimization**. Agents can separately analyze traffic, distance, delivery priority, and vehicle availability. The coordinator then selects an efficient route.

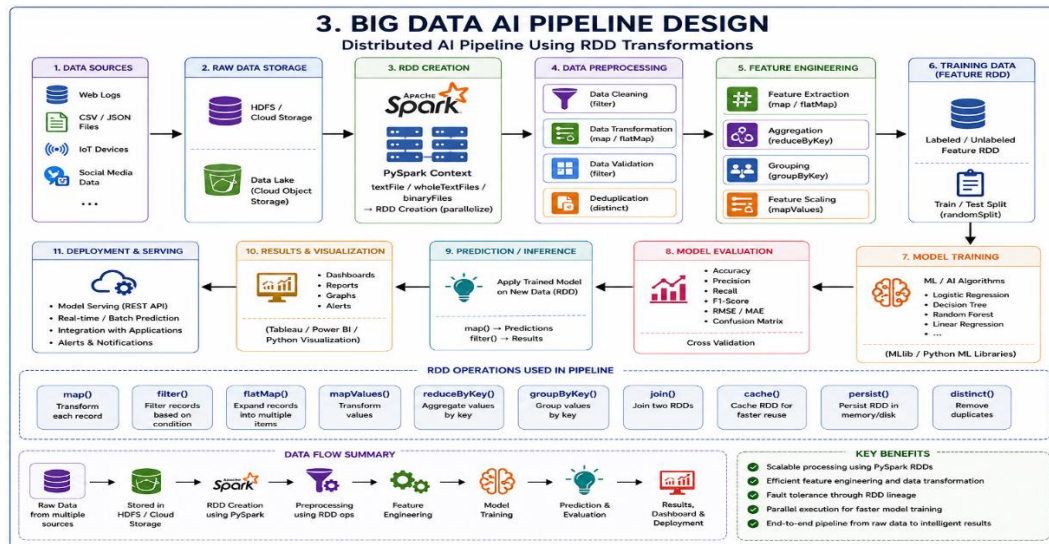
Expected Result

The combination of distributed PySpark processing and collaborative agents reduces processing time and allows complex optimization problems to be handled efficiently.

3. Big Data AI Pipeline Design

Proposed Solution: Distributed Customer Prediction Pipeline

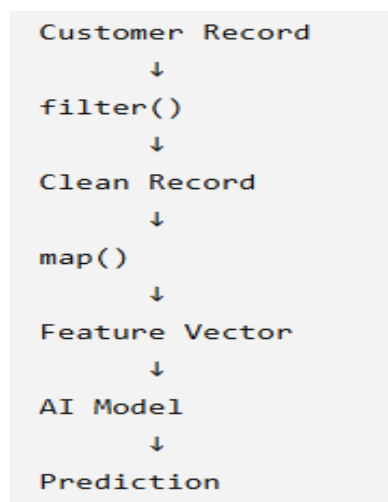
The system processes a large customer dataset using PySpark RDD transformations and trains an AI model to predict customer behaviour



Implementation Flow

First, the dataset is loaded into Spark and converted into an RDD. Invalid and incomplete records are removed using filter(). The map() transformation converts the remaining records into numerical features.

For example:



The reduceByKey() operation can be used to calculate customer statistics such as total purchases or transaction counts. The processed features are then supplied to a machine-learning model.

Fault Tolerance

RDDs maintain lineage information about the transformations used to create each partition. If a partition is lost because of a worker failure, Spark can recompute that partition from the original data instead of processing the complete dataset again.

Scalability

The system can process increasing amounts of data by adding more worker nodes to the Spark cluster. Since RDD operations are distributed, multiple partitions can be processed simultaneously.

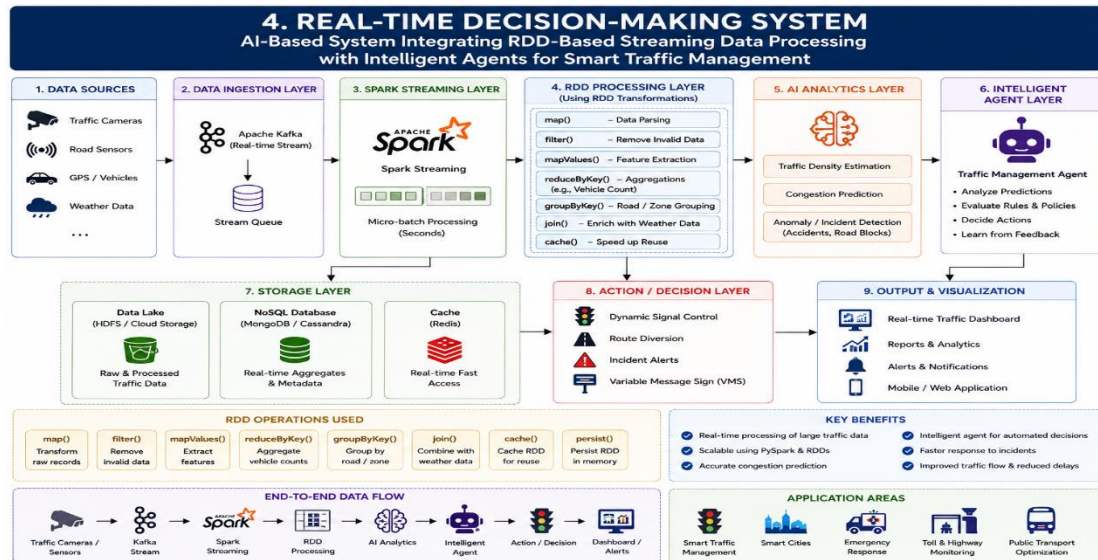
Expected Result

The proposed pipeline performs preprocessing, feature engineering, AI model training, and prediction in a distributed environment. It provides scalability, parallel processing, and fault tolerance

4. Real-Time Decision-Making System

Proposed Solution: Smart Traffic Monitoring System

The proposed system analyzes traffic information continuously and uses an intelligent agent to make real-time traffic decisions.



Working

1. Traffic sensors continuously generate vehicle and traffic-density information.
2. The streaming layer receives the incoming data.
3. PySpark processes the data in small batches.
4. RDD operations clean and transform the records.
5. Traffic density and vehicle counts are calculated.
6. The AI model predicts congestion levels.
7. The intelligent agent evaluates the prediction.
8. The agent recommends actions such as changing signal timing or generating a congestion alert.

Example Decision

```
graph TD; A[Vehicle Count = High] --> B[Traffic Density = High]; B --> C[AI Prediction = Congestion]; C --> D[Traffic Agent]; D --> E[Increase Green-Signal Duration]; E --> F[Reduce Traffic Waiting Time];
```

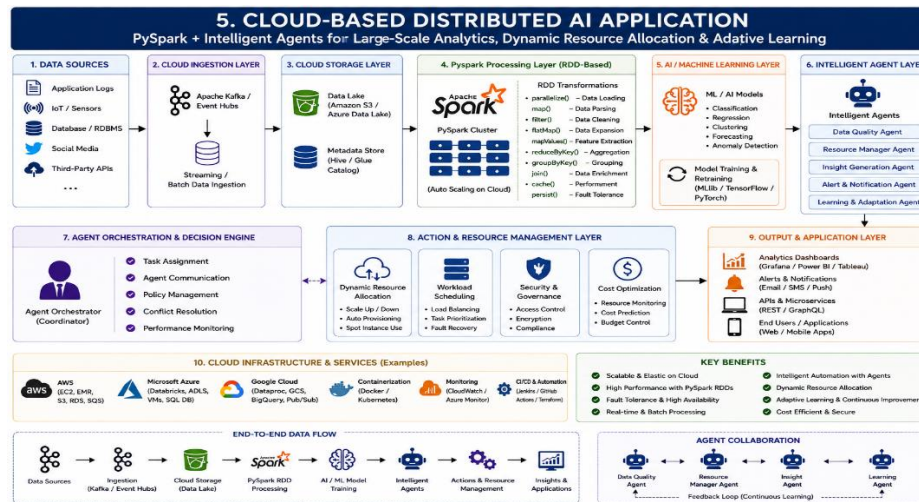
Vehicle Count = High
↓
Traffic Density = High
↓
AI Prediction = Congestion
↓
Traffic Agent
↓
Increase Green-Signal Duration
↓
Reduce Traffic Waiting Time

Expected Result

The system can continuously analyze traffic conditions and provide faster decisions than manual monitoring. It can help reduce congestion and improve traffic management.

5. Cloud-Based Distributed AI Application

This system uses cloud infrastructure, PySpark, and intelligent agents to analyze workload information and dynamically allocate computing resources.



Working

1. Application workload data is stored in cloud storage.
2. PySpark loads the data into distributed RDDs.
3. RDD transformations clean and organize the workload information.
4. Feature engineering identifies CPU usage, memory usage, request rate, and workload patterns.
5. The AI model predicts future resource requirements.
6. The resource-management agent monitors the predictions.
7. The agent increases or decreases resources according to workload.
8. The system continuously learns from new workload information.

Adaptive Learning

The system records previous workload and resource-usage information. This historical information can be used to improve future predictions and make resource allocation more efficient.

Expected Result

The cloud-based system dynamically manages resources according to workload requirements. It reduces unnecessary resource usage while maintaining application performance.